```
GNU Octave, version 3.2.2
Copyright (C) 2009 John W. Eaton and others.
This is free software; see the source code
for copying conditions.
.
.
```

...and finally the Octave prompt:

```
octave:1>
```

So, we are using Octave 3.2.2. You can *quit* or *exit* by typing in the same—not that I want you to quit; we have only just started. A lot of what Octave provides us revolves around manipulating matrices, so let's start off.

Define a simple 2x2 matrix, X:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad (1)$$

This is how we can define it in Octave:

```
octave:1> x= [1,2;3,4] x = 1 2 3 4
```

...and let's define another, Y:

$$\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \quad (2)$$

```
octave:1> Y= [5,6;7,8] Y = 5 6 7 8
```

We have two square matrices, X and Y. What shall we do? Let's find the product:

```
octave:13> X*Y ans = 19 22 43 50
```

Easy, isn't it? We defined two matrices, X and Y, and then found the product by simply X*Y. It's like how you would multiply two integers or real numbers. You get the idea, right? The individual columns are separate by a ',' and the rows by a ';'. Let's move on.

Things to try out

We have X and Y defined now. Let's try something like, Z=[X,Y] in Octave. What do you see? You should see the X matrix augmented or combined with the Y matrix:

```
Z =
   1  2  5  6
   3  4  7  8
```

We have so far had only real numbers in our matrices. How about complex numbers? What does Octave have to say about them? Let's see.

```
octave:18> XC = [1+2i,3;3+2i,4]
XC =

 1 + 2i  3 + 0i
 3 + 2i  4 + 0i

octave:19> YC = [1+2i,3;3+2i,4]
YC =

 1 + 2i  3 + 0i
 3 + 2i  4 + 0i

octave:20> XC*YC
ans =

  6 + 10i  15 + 6i
  11 + 16i  25 + 6i
```

Great! Octave handles them well too. As you can see, when you define some elements of a matrix as complex, the real elements are also written in complex form.

Now that we have had a taste of Octave, let us try to understand some basics, and after that we shall resume our fun with matrices on a more serious note.

Built-in data objects

We have already used numeric data objects in our matrices. Besides numeric data objects, we also have string data and data structure objects.

- **Numeric data objects:** Octave's built-in numeric objects include real, complex and integer scalars and matrices. All built-in floating point numeric data is currently stored as double precision numbers. For the exact values for the maximum and minimum possible real numbers that can be represented on your system, type *realmax* and *realmin*, respectively.
- **String data objects:** A character string in Octave consists of a sequence of characters enclosed in either double or single quotations. Strings won't be useful to us for the purpose of this article, so I won't talk about them.
- **Structure objects:** You can also have C style structures in Octave. You can have a number, a matrix and a string, and combine them under a common structure object. For example:

```
octave:4> x.a=4
x =
{
  a = 4
}

octave:5> x.a=4;
octave:6> x.b="foo baz"
x =
{
  a = 4
  b = foo baz
}

octave:7> x.b="foo baz";
octave:8> x
x =
{
  a = 4
  b = foo baz
}
```

Numeric data types

We can use the numeric objects to create higher level numeric data types like matrices, ranges and cell arrays. We've worked a bit with matrices and we will do a lot more with them now. I shall introduce the others later on in this series of series.

Let's now have some more matrix fun, shall we?

Determinants, inverses and singularity

For anyone who has taken a basic linear algebra course, close on the heels of matrices follows the concept of determinants. Let's try this out in Octave: